

SAMSUNG PRISM GENAI HACKATHON 3.0 · THEME 02

Smart Guided Troubleshooting Engine

A vague Galaxy complaint in. A validated, one-tap troubleshooting plan out.

13.7 ms

cache hit, P95

4.4 s

live plan, P95

0

URL leaks

\$0.00

per query

The problem

Turn a complaint into a plan an agent or the user can follow immediately

What users actually write

“My Galaxy S22 screen inputs are delayed and the touch responsiveness is laggy.”

“My screen got cracked when I dropped it. Also, every tap takes ages to register.”

“My battery dies by lunchtime even though I barely touch my phone.”

Optionally with a knowledge-base article: then the plan must be built from that article alone.

What every answer must do

1

Valid JSON in the given schema

every response, no exceptions

2

Steps only from the support article

nothing invented

3

Deeplinks copied from the catalog

never synthesised by a model

4

No web URLs anywhere

even if the article contains one

5

Fast

P95 $\leq$ 300 ms cached, $\leq$ 8 s live

Three rules the design never breaks

Each rule turns a failure mode into something that cannot happen

1

The model never writes user-facing text

It selects step numbers from the article; code copies those steps verbatim.

A step that is not in the source cannot appear.

2

The model never sees or writes a deeplink

It describes the screen it wants; a BM25 + dense retriever finds the catalog entry.

An invented URI is structurally impossible.

3

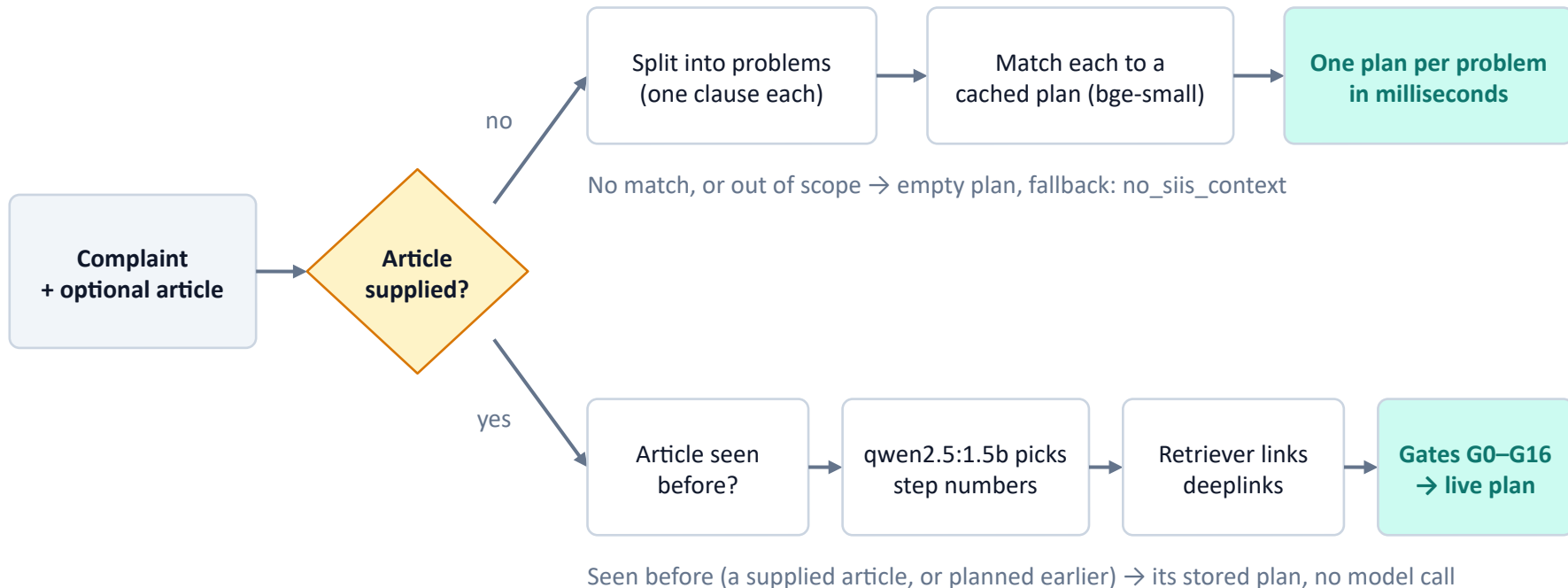
The cache is the pipeline, frozen

Cached plans are the same pipeline's output, run once per article at build time.

No hand-written answer path to drift or hard-code.

How a request is handled

POST /v1/troubleshoot → one of three paths; every plan returned has passed the gates



Build time: the same pipeline ran once per supplied article with Claude Opus 5; the validated plans and their embeddings are frozen in artifacts/. No hosted model is called at runtime.

The cache path: meaning, not strings

11 validated plans, 130 cached phrasings, no model call

13.7 ms

P95 for a phrasing never
seen before (0.1 ms exact)

84.6%

hit rate on 26 tuning
paraphrases · 100% on 27
written afterwards

340/340

complaints naming two
known problems got both
plans

37/38

held-out out-of-scope
complaints correctly fell
back

How it works

- Two-factor acceptance: a match must be similar enough, and either very close or clearly ahead of the runner-up plan.
- Multi-intent: split only at explicit boundaries (sentence ends, “also”, “, and the...”), each part judged like a whole query.
- Out-of-scope anchors: topics no plan covers (battery, connectivity...) pull a complaint to a fallback instead of the nearest display plan.
- Thresholds tuned on one set, reported on a test set written before evaluation.

The live path: a 1.5B model on a laptop GPU

For a new article: plan it in seconds, locally, with nothing invented

1

Pick

gwen2.5:1.5b selects step numbers and groups them into actions, under a JSON-schema grammar.

2

Link

Each settings screen is resolved against the 578-entry catalog by hybrid retrieval.

3

Order

Settings screens first, hands-on checks next, destructive steps like a factory reset last.

4

Validate

Gates G0–G16: schema, URL scrub, catalog URIs, one action = one screen; repaired or rejected.

2.6 s

median (P50)

4.4 s

P95, against an 8 s budget

33/33

live plans pass every gate

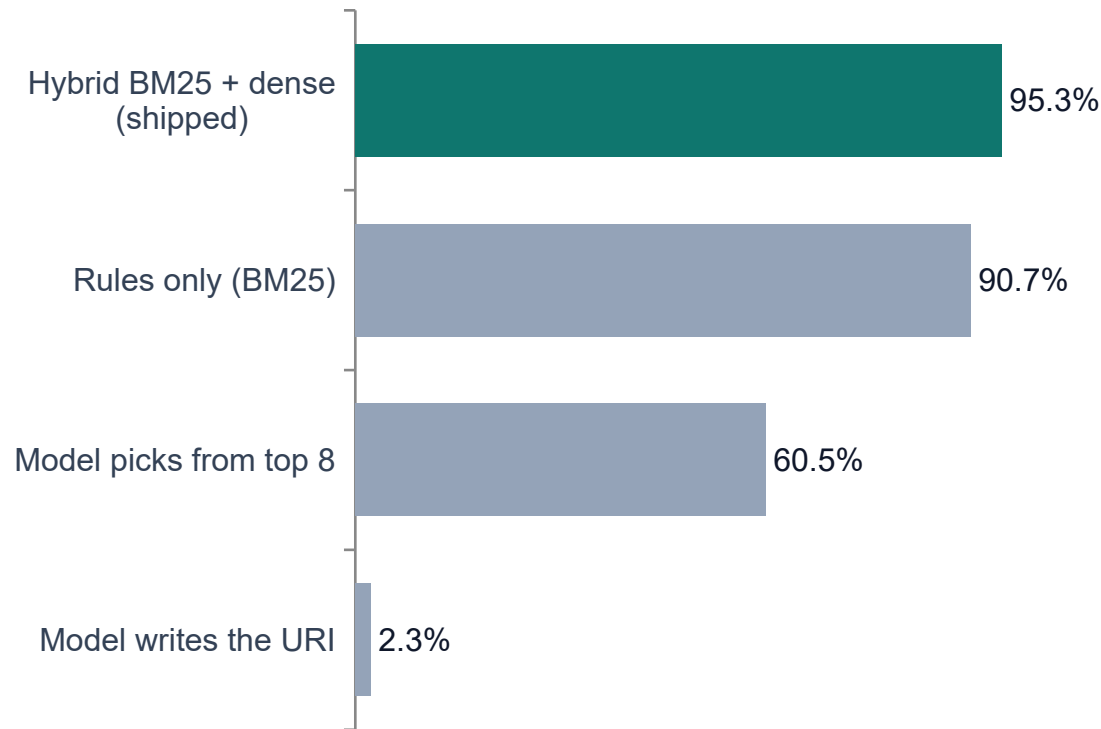
\$0.00

~880 tokens, local inference

The first validated plan for an article is kept, so repeats are identical and instant.

Why a retriever, not the model, picks deeplinks

Precision@1 on 43 hand-labelled Display screens

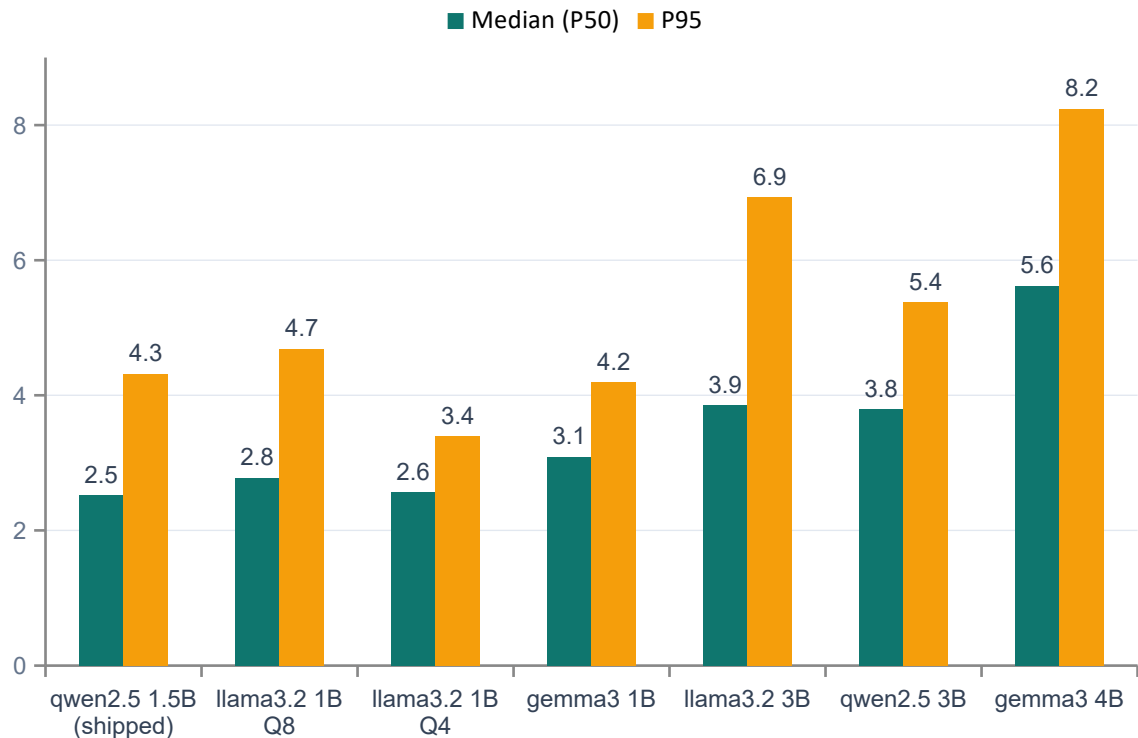


What else the ablation showed

- Masked URIs are opaque, so a model writing them invents 42 of 43.
- Hybrid takes 16.9 ms per screen (P95), against 279 ms for a model call.
- On free-form wording, hybrid recall@1 is 66.7% vs 13.3% for rules alone.
- 100% on 26 Battery, Camera and Performance screens.

Choosing the local model: seven measured

Same 11 articles, 3 passes each, one run on the final code (seconds)



Latency moves 5–10% between runs; the dedicated benchmark measured qwen2.5:1.5b at 4.4 s P95.

Why qwen2.5:1.5b

- Fastest median, narrowly: the 4-bit llama3.2 1B is within 2%.
- 33/33 plans pass every gate; gemma3 1B and 4B do not.
- Best step accuracy: 2.97 of 3.
- The 3B models link more screens but cost about 1.3 s more at the median.
- The 4-bit llama3.2 1B is as fast (lowest P95) but loses on quality: 63 of 72 action names replaced, step accuracy 2.87.

Results against the problem statement's targets

Full report in Appendix C format: docs/metrics.md

Measure	Target	Measured
Cache hit, P95 latency	≤ 300 ms	0.1 ms exact · 13.7 ms new phrasing
Live plan, P95 latency	≤ 8 s	4.4 s (median 2.6 s)
Schema-valid responses	$\geq 99\%$	100% (325/325 over HTTP, 20/20 batch)
Web-URL leaks	0	0, including planted URLs
Deeplinks from the catalog	100%	100%, no model writes one
Cache hit rate, unseen phrasings	$\geq 80\%$	84.6% tuning · 100% held-out
Step accuracy (0–3)	rubric	2.92 cached · 2.97 live
Deeplink relevance (0–2)	rubric	1.78
Cost per query	tracked	\$0.00, local inference

Measured on one laptop (i5-13420H, RTX 4050 6 GB) with nothing else running.

The demo

python scripts/demo.py → http://127.0.0.1:8000/demo

The screenshot shows a web application titled "Smart Guided Troubleshooting Engine" with a subtitle "A vague complaint in a validated one-tap troubleshooting plan out". At the top, there are status indicators: "Ready", "11 cached plans", "130 cached phrases", "578 catalog deepinks", "Local model: qwen2.5:1.5b", and a "Theme" button. The interface is divided into several sections:

- DEMO SCENES:** A list of seven scenarios, each with a brief description. The first scenario, "Known issue", is highlighted with a blue border.
- CUSTOMER COMPLAINT:** A text box containing the user's complaint: "My Galaxy S22 screen inputs are delayed and the touch responsiveness is laggy, causing a noticeable delay when I try to interact with the phone."
- KNOWLEDGE-BASE ARTICLE (OPTIONAL):** A section with the text "None — rely on the cache" and a dropdown arrow.
- Build troubleshooting plan:** A blue button with the text "Build troubleshooting plan" and a keyboard shortcut "Ctrl + Enter · POST /v1/troubleshoot" below it.
- PATH:** A section showing the current path: "Cache hit", "no model call".
- LATENCY:** A section showing the latency: "12 ms", "server · 10 ms round trip".
- TOKENS:** A section showing the token count: "0", "0 in · 0 out".
- COST:** A section showing the cost: "\$0.00", "local inference".
- TOUCH RESPONSE DELAY:** A section with a "Confidence: 100%" bar and the instruction "Follow these steps to perform this Touchscreen Response Troubleshooting". Below this, there are four numbered steps, each with a description and a "Settings screen" button.

Seven scenes

- 1 Known issue: cache hit in ms
- 2 Same issue, new words
- 3 Two problems: two plans
- 4 Never-seen issue: live plan
- 5 Out of scope: safe fallback
- 6 Planted URL: 0 leaks
- 7 Spare: camera (live)

What it does not do yet

Measured limits, from docs/metrics.md §6

Limitations

- Cached plans cover Display only: all supplied data is Display.
- Live plans link a specific screen for 19% of settings actions (69% cached).
- In a two-problem complaint, a problem with no plan is dropped silently.
- A new article always takes the live path, even if the complaint matches a cached plan.

Next steps

- Compile plans for Battery, Camera and Performance articles.
- Tell the caller when part of a complaint went unanswered (needs a meta key).
- Decide whether live plans may be shared with other callers' similar queries.
- Raise live-path deeplink coverage with better screen descriptions.

Try it

```
pip install -r requirements.txt  
ollama pull qwen2.5:1.5b  
python scripts/demo.py
```

github.com/Sanyam-Ahuja/prism

Tag PRISM_GENAI_HACKATHON_Y2026 · README, metrics report, AI disclosure and demo runbook inside